WEB APPLICATION · NODE.JS

Secure access, managed with care.

A responsive account portal with role-based dashboards, administrator-reviewed sign-up, email notices, digital signatures, and encrypted database backups.

Implementation and local workflow verified

Project at a glance

3
Account roles: administrator, staff, and viewer.

12+
Minimum password length, with peppered bcrypt hashing.

7 days
Encrypted local daily snapshots are retained before rotation.

P-256
Browser-held ECDSA keys support signed statements.

What the application provides

One Node.js service delivers the web interface and JSON API. SQLite supports a single persistent app instance; an optional Turso mode supports a remote database. The responsive interface separates the team directory and approval tools from regular member views.

New-account journey

User requests access › **Admin reviews request** › **Approved user signs in**

ACCOUNTS AND WORKFLOW

A clear path from request to access

Public account requests start with viewer access in a pending state. The applicant cannot sign in until an administrator approves the request. Declined requests remain blocked.

Capability	Implemented behavior
Admin dashboard	Review pending requests, view approved members, create accounts, and remove other accounts.
Approval controls	Approve or decline sign-up requests. The final administrator and the currently signed-in account are protected from deletion.
Member dashboard	Shows each user's role and account details, with personal email and signature settings.
Access rules	Server-side role checks protect administrator actions and directory data; interface visibility alone is not the security boundary.
Optional Google sign-in	Google identity tokens are verified server-side. New Google users follow the same pending approval process when configured.

Email notifications

When access is requested

The applicant receives a request receipt; the configured administrator email receives a review notice.

After administrator review

The applicant receives an approval or decline notice. Approval notices include the configured app URL.

> **Provider setup required:** live delivery needs a Resend API key, a verified sender address, an administrator email address, and the public app URL in the private environment configuration. Those credentials were not supplied for this verification, so delivery was tested with a mock Resend response.

Admin-managed accounts

Administrators can add an account with a selected role and temporary password, and delete other accounts. New passwords should be shared securely. Self-deletion and deletion of the last administrator are blocked.

OPERATIONS AND ASSURANCE

Protect data and make recovery possible

Password protection

HMAC-SHA-256 password pre-hashing with a server-side pepper, followed by bcrypt at cost 12. Production remote-database deployments require a stable AUTH_PEPPER.

Session protection

Signed, HttpOnly, SameSite cookies; HKDF-derived session signing key; CSRF checks on state-changing API requests; Secure cookies in production.

Digital signatures

ECDSA P-256 signing uses the browser Web Crypto API. Private signing keys stay in browser IndexedDB; the server stores public keys.

Encrypted backups

Local SQLite snapshots are encrypted with AES-256-GCM, include authenticated integrity protection, and are kept for seven days. A separate backup key is required for recovery.

Verification performed

Check	Result
JavaScript and dependencies	Syntax checks passed for 11 JavaScript files; installed dependency tree resolved.
Account workflow	New request was blocked from login while pending, approved by admin, then allowed to sign in.
Administrative actions	Admin account creation and deletion both succeeded.
Email workflow	Three Resend messages were verified with a test mock: applicant receipt, admin alert, applicant approval notice. No live email credentials were configured.
Backup recovery	Encrypted SQLite backup decrypted successfully; SQLite integrity check returned “ok”.
Local preview	Web page and API returned HTTP 200 at 127.0.0.1:8000.

> **Run locally:** Node.js 20+ and npm. Start with `npm start` and open the Node server URL, not a standalone HTML file. Local demo data is stored in the ignored .local-data folder.

Deployment notes

Production needs HTTPS, configured private environment variables, and persistent SQLite storage or a configured Turso database. Temporary tunnel links are for demos and expire when their tunnel process stops.

A GitHub remote and public production host have not been configured from this workspace.